

Two-Wheeler Event Detection and Safety Scoring

A Real-Time, Multi-Sensor Approach to Riding Event Detection

Bisleri - 4

Naman Aggarwal - 204101018

Samarth Singla - 204101020

Shivek Gupta - 204101011

Vansh Goyal - 204101044

December 2, 2025

Chapter 1

Introduction

1.1 Problem Statement

Develop an embedded system to capture the vehicle dynamics using an Inertial Measurement Unit (IMU). Place it in a bike and collect data with various events. Further, use the data to develop learning algorithms for classification of the events.



Figure 1.1: Data Collection Node Deployed on Vehicle

1.2 Core Problem

Two-wheeler fleet operators, such as delivery services and bike taxis, face significant challenges due to a lack of objective data on how their vehicles are being used. These challenges include:

- **Difficulty in tracking rider behaviour:** Without real-time monitoring capabilities, fleet managers cannot effectively observe and evaluate how riders operate their vehicles in various conditions.
- **Ensuring driver safety:** The absence of comprehensive safety monitoring systems makes it challenging to identify risky driving patterns and prevent accidents before they occur.
- **Managing maintenance costs:** Unpredictable maintenance expenses arise from the inability to track vehicle stress patterns, harsh usage, and road hazard encounters that accelerate wear and tear.
- **Tracking rider behaviour around road hazards:** Monitoring how riders navigate potholes, speed bumps, and other road obstacles is difficult without dedicated sensor systems and real-time analysis.
- **Generating rider quality scores:** The lack of objective metrics makes it impossible to create fair, data-driven ratings that reflect actual driving quality and can be used for performance evaluation, insurance premiums, or incentive programs.

These problems collectively result in increased operational costs, compromised safety standards, and inefficient fleet management practices that hinder the growth and sustainability of two-wheeler fleet operations.

1.3 Motivation

The primary motivation for this project is to create a comprehensive, intelligent system that enhances two-wheeler safety and enables efficient fleet management by addressing critical gaps in current technology. Modern transportation ecosystems, particularly in urban environments, rely heavily on two-wheeler fleets for last-mile delivery and ride-hailing services. However, these operations face significant challenges in monitoring vehicle usage, ensuring rider safety, and optimizing maintenance schedules. The absence of objective, real-time data about rider behavior and vehicle dynamics creates blind spots that can lead to:

- Increased accident rates due to undetected risky driving patterns
- Higher maintenance costs from unpredictable vehicle wear
- Inefficient fleet utilization and resource allocation
- Difficulty in implementing fair, data-driven rider evaluation systems

Recent advances in embedded systems, sensor technology, and machine learning have made it feasible to develop sophisticated monitoring solutions that can run on affordable, compact hardware. By leveraging high-frequency sensor data, intelligent algorithms, and cloud connectivity, we can transform how two-wheeler fleets are managed and how rider safety is ensured. This project aims to bridge the gap between cutting-edge technology and practical fleet management needs, creating a system that not only detects and classifies driving events but also provides actionable insights that can improve safety, reduce costs, and enhance operational efficiency.

1.4 Our Solution

We present an end-to-end embedded system designed to revolutionize two-wheeler fleet management and rider safety through intelligent, real-time monitoring and analysis. Our solution encompasses the following key components:

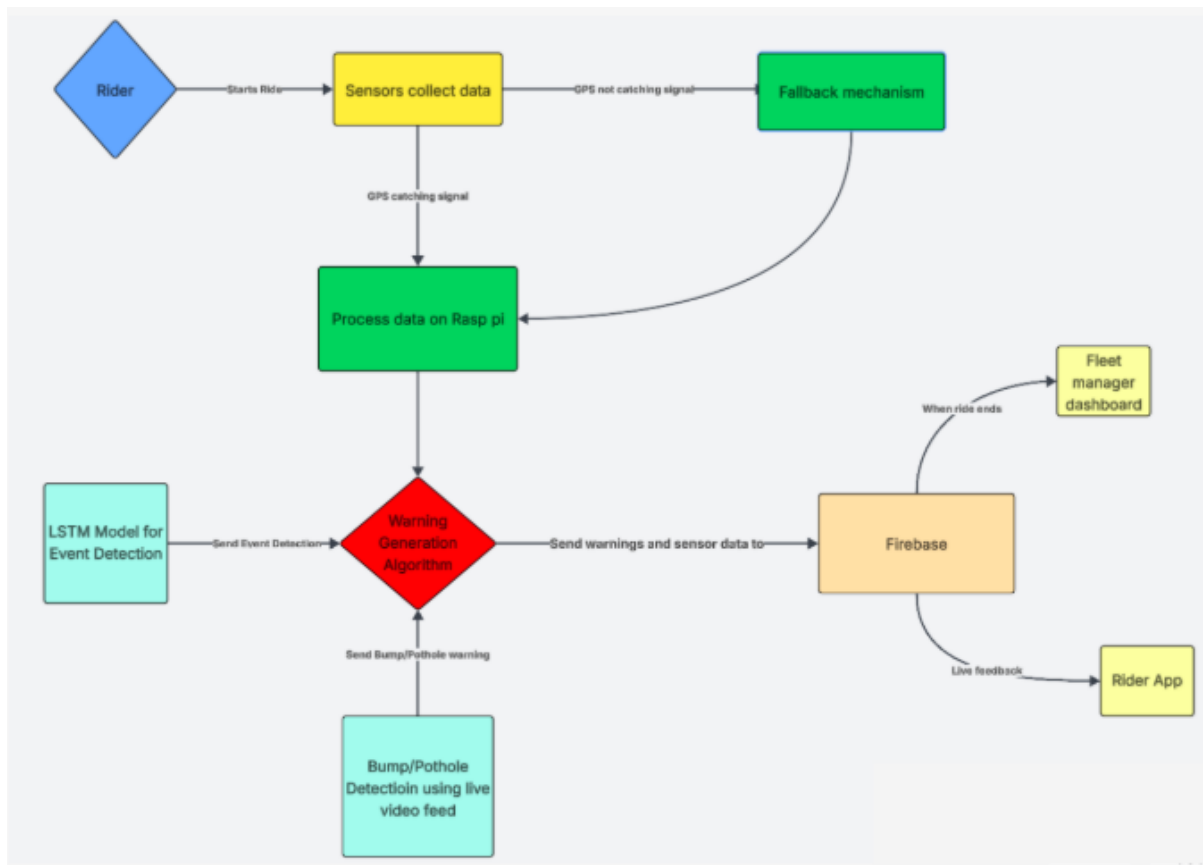


Figure 1.2: System Architecture Diagram

1.4.1 System Architecture

- **Embedded Hardware Platform:** A complete system designed to run on a Raspberry Pi, making it affordable, compact, and easy to deploy across entire fleets. The system integrates multiple sensors and processing modules into a single, robust unit.



Figure 1.3: Deployed Node

- **High-Frequency Data Acquisition:** Captures sensor data at up to 1000 Hz from the MPU6500 IMU, with synchronized GPS readings and camera feeds, ensuring no critical event goes undetected.
- **Real-Time Event Detection and Classification:** Employs a hybrid approach combining:
 - LSTM neural networks for complex event recognition (bumps, turns, stops)
 - Threshold-based algorithms for instantaneous warnings (overspeeding, harsh braking)
 - Computer vision models for road hazard detection (potholes, speed bumps)
- **Intelligent Warning System:** Provides immediate audio/visual alerts to drivers about:
 - Potential hazards ahead (potholes, speed bumps)
 - Dangerous driving behaviors (speeding, harsh braking, aggressive acceleration)
 - Sharp turns at high speeds
- **Real-Time Feedback and Cloud Integration:** Continuously uploads critical data to Firebase, enabling:
 - Live monitoring of fleet status
 - Immediate access to current speed and active warnings
 - Seamless integration with mobile applications

- **Comprehensive Behavior Tracking:** Records every aspect of rider behavior throughout the journey:
 - Acceleration and braking patterns
 - Speed compliance with posted limits
 - Handling of road obstacles
 - Turning behavior and cornering speeds
- **Automated Ride Reports:** Generates detailed, data-driven reports at the end of each ride, including:
 - Event timeline and frequency
 - Safety score and driving quality metrics
 - Route visualization with hazard markers
 - Comparative analysis against fleet averages
- **Rider Mobile Application:** A dedicated smartphone app that provides:
 - Real-time speed and warning displays
 - Live ride statistics and performance metrics
 - Historical ride data and trend analysis
 - Gamification elements to encourage safe driving
- **Fleet Manager Dashboard:** A comprehensive web-based platform offering:
 - Fleet-wide analytics and performance overview
 - Individual rider profiles with ride-wise statistics
 - Comparative rider rankings based on safety scores
 - Maintenance prediction and cost optimization insights
 - Customizable alert thresholds and reporting parameters

1.4.2 Key Benefits

This integrated solution delivers tangible value across multiple dimensions:

1. **Enhanced Safety:** Real-time warnings and continuous monitoring significantly reduce accident risks by alerting riders to dangerous behaviors and road hazards immediately.
2. **Objective Performance Metrics:** Data-driven scoring systems eliminate subjective bias in rider evaluation, creating fair and transparent performance assessments.
3. **Predictive Maintenance:** By tracking vehicle stress patterns and road hazard encounters, the system enables proactive maintenance scheduling, reducing unexpected breakdowns and repair costs.

4. **Operational Efficiency:** Fleet managers gain unprecedented visibility into operations, enabling better resource allocation, route optimization, and policy enforcement.
5. **Scalability:** The system architecture supports easy deployment across fleets of any size, with centralized cloud infrastructure handling data from thousands of devices.

By combining embedded hardware, advanced algorithms, and cloud connectivity, our solution transforms raw sensor data into actionable insights that improve safety, reduce costs, and enhance the overall efficiency of two-wheeler fleet operations.

Chapter 2

Data Collection and Calibration

2.1 Data Acquisition Architecture

The data acquisition core is designed for high-throughput, concurrent data streams without blocking the main processing loop. This architecture ensures that sensor data is continuously collected and made available for real-time analysis.

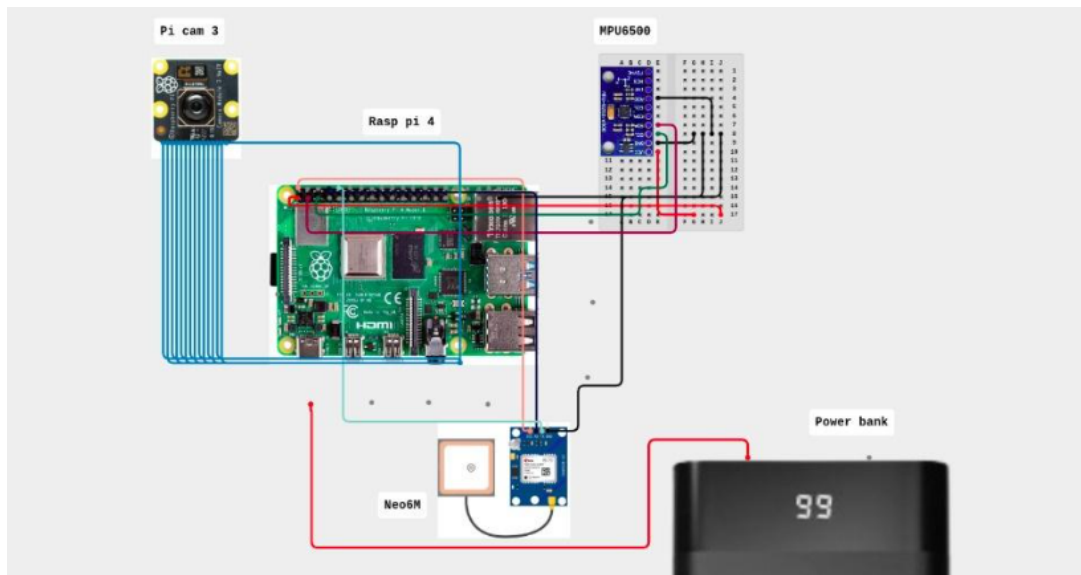


Figure 2.1: Hardware Circuit Diagram and Connections

2.1.1 Multi-Threading Architecture

Each sensor and I/O task runs in a dedicated thread to ensure parallel operation. This design prevents any single sensor from blocking the data collection pipeline:

- **MPU Thread:** Continuously reads accelerometer and gyroscope data at 1000 Hz
- **GPS Thread:** Monitors GPS module for location and speed updates
- **Speed Limit Thread:** Retrieves geofenced speed limits based on current location
- **Main Sampling Loop:** Orchestrates data collection at a consistent 104 Hz

2.1.2 Shared Variables and Synchronization

Sensor threads update global variables that are protected by locks to prevent race conditions, ensuring atomic reads and writes. The key shared variables include:

```
1 # Protected by data_lock
2 latest_mpu = {
3     'accel_x': 0.0,
4     'accel_y': 0.0,
5     'accel_z': 0.0,
6     'gyro_x': 0.0,
7     'gyro_y': 0.0,
8     'gyro_z': 0.0
9 }
10
11 latest_gps = {
12     'latitude': 0.0,
13     'longitude': 0.0,
14     'speed': 0.0
15 }
16
17 speed_limit = 0.0
```

2.1.3 Sampling Loop

The main sampling loop operates at a consistent 104 Hz frequency. At each time step, it samples the latest data from the shared variables to create a unified data point that combines all sensor readings with precise timestamps. This synchronized data stream is then written to shared memory for consumption by the warning generation system.

2.2 MPU6500 Calibration and Filtering

The MPU6500 6-axis Inertial Measurement Unit (IMU) provides accelerometer and gyroscope data. To ensure accurate measurements, a two-stage filtering process is employed.

2.2.1 Initial Calibration

On startup, the system performs a calibration by calculating zero bias through averaging sensor readings while the vehicle is stationary:

1. The system collects sensor readings for approximately 1 second
2. Mean values are calculated for all six axes
3. These mean values represent the inherent sensor bias
4. The bias is subtracted from all subsequent readings to correct for inherent sensor error

This calibration compensates for manufacturing variations and temperature-dependent offsets in the sensor.

2.2.2 Real-Time Kalman Filtering

After bias removal, a 1D Kalman Filter is applied independently to each of the 6 axes (3 accelerometer + 3 gyroscope). The Kalman filter intelligently smooths the signal by combining predictions with noisy measurements.

Filter Parameters:

- Process noise covariance (Q): 1×10^{-5}
- Measurement noise covariance (R):
 - Accelerometer: 1×10^{-2}
 - Gyroscope: 5×10^{-3}

The Kalman filter operates in two steps:

Prediction Step:

$$\begin{aligned}\hat{x}_k^- &= \hat{x}_{k-1} \\ P_k^- &= P_{k-1} + Q\end{aligned}$$

Update Step:

$$\begin{aligned}K_k &= \frac{P_k^-}{P_k^- + R} \\ \hat{x}_k &= \hat{x}_k^- + K_k(z_k - \hat{x}_k^-) \\ P_k &= (1 - K_k)P_k^-\end{aligned}$$

where $\hat{x}_k$ is the estimated state, P_k is the error covariance, K_k is the Kalman gain, and z_k is the measurement.

This filtering approach reduces sensor jitter and noise while preserving true motion, providing a clean, reliable signal for downstream processing.

2.3 GPS Speed Calibration

A hybrid approach is employed to guarantee a reliable speed value at all times, even with intermittent GPS signal.

2.3.1 GPS as Primary Source

When a valid GPS signal for speed is available, it is used as the ground truth for vehicle speed. GPS speed measurements are considered reliable when:

- The GPS module has a valid fix (sufficient satellite visibility)
- The reported speed is ≥ 0.5 km/hr (above noise threshold)

2.3.2 Accelerometer-Based Fallback Mechanism

The fallback mechanism is triggered when:

- GPS signal is lost (tunnels, urban canyons, dense foliage)
- GPS reports a speed of < 0.5 km/hr

Calibrated Integration Method

The system uses a calibrated formula to integrate accelerometer data into speed estimates:

$$v_{k+1} = v_k + (a_x - \text{BIAS}) \cdot \Delta t \quad (2.1)$$

where:

- v_k is the current speed estimate
- a_x is the measured longitudinal acceleration (in g)
- Δt is the time step (seconds)
- BIAS is the systematic accelerometer offset

Calibration Parameters

The calibration parameters were determined empirically by analyzing 32,984 data points of simultaneous GPS and accelerometer measurements:

- **OPTIMAL_BIAS** = 0.117588g: Systematic offset determined through least-squares optimization against real-world GPS speed data
- **DEADBAND** = 0.02g: Threshold applied to ignore noise and prevent drift when the vehicle is stationary
- **RMSE** = 22.67 km/h: Root mean square error between calibrated accelerometer speed and GPS ground truth

Deadband Application

To prevent speed drift due to sensor noise when the vehicle is stationary:

```
1 accel_corrected = accel_x - OPTIMAL_BIAS
2 if abs(accel_corrected) < DEADBAND_G:
3     accel_corrected = 0.0
4 speed += accel_corrected * G_TO_MS2 * dt
5 speed = max(0.0, speed) # Prevent negative speeds
```

This hybrid GPS-accelerometer approach ensures continuous, reliable speed estimation regardless of GPS signal availability.

Chapter 3

Warning Generation Algorithm

3.1 Overview

The warning generation system operates as a separate process that continuously monitors sensor data and generates real-time alerts for dangerous driving behaviors and road hazards. The system is built on a multi-threaded architecture with inter-process communication to ensure efficient, parallel processing of multiple warning conditions.

3.2 Inter-Process Communication

Data flows from the acquisition system (`main2.py`) to the warning generation system (`Warning_Generate.py`) using shared memory for zero-copy, high-performance communication. The shared memory bridge transfers batches of sensor data at 1 Hz, containing 104 data points per batch with synchronized timestamps and sensor readings.

3.3 Multi-Threading Architecture

The warning generation system employs eight dedicated threads running in parallel:

1. **Shared Memory Reader Thread:** Reads batches from shared memory
2. **LSTM Prediction Thread:** Classifies driving events
3. **Overspeeding Detection Thread:** Monitors speed limit compliance
4. **Speedy Turns Detection Thread:** Identifies dangerous cornering
5. **Harsh Braking Detection Thread:** Detects abrupt deceleration
6. **Sudden Acceleration Detection Thread:** Identifies aggressive acceleration
7. **Camera Warning Reader Thread:** Processes road hazard alerts
8. **Firebase Push Thread:** Uploads data to cloud database

3.4 Warning Detection Algorithms

3.4.1 Overspeeding Detection

Continuously checks if the current speed exceeds the speed limit fetched from the OLA Maps Speed Limit API. When a violation is detected, an immediate warning is generated.

3.4.2 Speedy Turns Detection

The algorithm operates in three stages:

1. Checks if the LSTM model's latest prediction is 'LEFT' or 'RIGHT'
2. If a turn is detected, verifies:
 - Z-axis angular velocity (gyro_z) > SPEEDY_TURN_THRESHOLD
 - Current speed > 20 km/h
3. If all three conditions are met, a "Speedy Turn" warning is flagged

3.4.3 Harsh Braking Detection

Detects sharp, negative changes in forward acceleration by analyzing the rate of change:

1. Calculates jerk (derivative of acceleration) by finding $\frac{d(\text{accel}_x)}{dt}$ over the batch
2. Checks if minimum jerk or average jerk < HARSH_BRAKE_THRESHOLD
3. Flags harsh braking event when threshold is exceeded

3.4.4 Sudden Acceleration Detection

Identifies sharp, positive changes in forward acceleration:

1. Calculates the slope of accel_x values over the batch
2. Checks if average slope or maximum instantaneous slope > SUDDEN_ACCEL_THRESHOLD
3. Flags sudden acceleration when threshold is exceeded

3.4.5 Pothole and Bump Detection

A separate computer vision module (`live_detect.py`) processes camera frames to detect road hazards. The script alternates between checking for potholes and bumps, writing its findings to a shared JSON file that the camera warning reader thread monitors for alerts.

3.4.6 Event Detection via LSTM

The LSTM neural network classifies driving events into five categories:

- **BUMP**: Vertical disturbance from road obstacles
- **LEFT**: Left turn maneuver
- **RIGHT**: Right turn maneuver
- **STOP**: Vehicle stopping or stopped state
- **STRAIGHT**: Normal straight-line driving

The model processes sequences of 104 timesteps with 6 features (3 accelerometer + 3 gyroscope axes) to capture temporal patterns in the sensor data.

Chapter 4

Pothole and Bump Detection

4.1 Overview

The pothole and bump detection system uses computer vision to identify road hazards in real-time, providing advance warnings to riders. The system operates independently through the `live_detect.py` script, which continuously processes camera frames and communicates findings to the warning generation system.

4.2 Detection Pipeline

4.2.1 Frame Capture and Processing

The `live_detect.py` script captures frames from the Raspberry Pi camera at approximately 1.5-2 Hz. Each captured frame undergoes the following steps:

1. Frame acquisition from the camera module
2. Inference using the Roboflow object detection model run locally on the Raspberry Pi
3. Analysis of detection results for "pothole" or "speedbump" objects
4. Update of the `camera_warnings.json` file with the latest status



Figure 4.1: Annotated Detection of a Pothole

The JSON file format is simple and efficient:

```
1 {"pothole": 1, "bump": 0} # Pothole detected
2 {"pothole": 0, "bump": 1} # Speed bump detected
3 {"pothole": 0, "bump": 0} # No hazards detected
```

This file is continuously monitored by the camera warning reader thread in the main warning generation system.

4.2.2 Evolution of Detection Approach

Previous Approach: The initial implementation used an alternating detection strategy:

- All even frames attempted to detect potholes
- All odd frames attempted to detect speed bumps

This approach required two separate inference passes and introduced latency in detecting both types of hazards.

Updated Approach: The system now utilizes a single, efficient object detection model that can detect both potholes and speed bumps in one frame simultaneously. This unified approach:

- Reduces computational overhead by eliminating redundant inference passes
- Enables simultaneous detection of multiple hazard types
- Improves overall system responsiveness

4.3 Object Detection Model

4.3.1 Model Architecture

The system uses the `potholes-and-speed-bumps-detection/1` model from Roboflow. Key characteristics include:

- **Single-pass detection:** Trained to recognize both pothole and bump classes in a single inference pass
- **Optimized for edge devices:** Model is quantized and optimized for efficient execution on Raspberry Pi hardware
- **Dual-class output:** Simultaneously detects and localizes both hazard types with bounding boxes and confidence scores

4.3.2 Camera Configuration

The system uses the `picamera2` library for camera access, providing native, low-latency communication with the Raspberry Pi camera module. **Camera Parameters:**

- **Resolution:** 320x240 pixels

- **Rationale:** Low resolution maximizes frame rate and reduces inference time, ensuring real-time performance without sacrificing detection accuracy for the task
- **Frame Rate:** Approximately 1.5-2 Hz (limited by inference time, not camera capabilities)

The low resolution is sufficient for road hazard detection as potholes and speed bumps present distinctive shapes and features even at reduced pixel density.

4.4 Performance Characteristics

Average Precision by Class (mAP50)



Figure 4.2: Accuracy Analysis of Pothole and Bump Detection

The computer vision module operates continuously in parallel with the main sensor acquisition system, providing timely hazard warnings without impacting the high-frequency data collection from the IMU and GPS modules. The detection latency (frame capture → inference → JSON update) is approximately 500-667 ms, providing adequate advance warning time for riders traveling at typical urban speeds.

Chapter 5

LSTM Model for Event Classification

5.1 Why LSTM?

The choice of Long Short-Term Memory (LSTM) networks over traditional machine learning models is fundamental to achieving accurate event classification in this system. The key distinction lies in how different models handle temporal data:

Traditional ML Models: Algorithms such as Decision Trees, Random Forests, and Support Vector Machines treat each data point independently. They cannot inherently capture the sequential nature of sensor data or understand how past measurements influence current observations.

LSTM Networks: A special type of Recurrent Neural Network (RNN) designed to capture long-term dependencies in sequential data. LSTMs explicitly model temporal dependencies through their unique cell architecture, which maintains a memory state that evolves over time. This makes them far more powerful for tasks involving sequences, such as recognizing driving events from continuous IMU data streams. For vehicle dynamics classification, where events like turns, bumps, and stops manifest as patterns across multiple consecutive timesteps, the temporal modeling capability of LSTMs is essential.

5.2 Model Architecture

5.2.1 Input Features

The LSTM model processes sequences of 104 consecutive timesteps, where each timestep contains 6 features from the MPU6500 IMU:

- **Accelerometer data:** $accel_x$, $accel_y$, $accel_z$ (3 axes)
- **Gyroscope data:** $gyro_x$, $gyro_y$, $gyro_z$ (3 axes)
- **Speed**

Input shape: (104, 7) - representing 104 timesteps with 7 features each. The 104-timestep window corresponds to exactly 1 second of data at the 104 Hz sampling rate, providing sufficient temporal context to distinguish between different driving events.

5.2.2 Output Features

The model outputs a probability distribution over 5 classes representing distinct driving events:

1. **BUMP**: Vertical disturbance from road obstacles
2. **LEFT**: Left turn maneuver
3. **RIGHT**: Right turn maneuver
4. **STOP**: Vehicle stopping or stopped state
5. **STRAIGHT**: Normal straight-line driving

Output shape: (5,) - a 5-dimensional softmax probability vector.

5.3 Training Process

5.3.1 Data Collection and Preparation

Dataset Size: Approximately 2.7 lakh (270,000) data points were collected from real-world riding sessions and manually labeled with corresponding event classes. **Data Splitting:** The dataset was divided into three subsets:

- **Training set:** 60% of data
- **Validation set:** 20% of data
- **Test set:** 20% of data

5.3.2 Handling Class Imbalance

A significant challenge in the dataset was class imbalance: the dataset contained far more STRAIGHT samples than any other class, as most riding time consists of straight-line travel. **Solution:** Class weights were employed during training. This technique adjusts the loss function to penalize the model more heavily for misclassifying minority classes (such as BUMP or STOP), forcing the model to learn these underrepresented patterns effectively rather than simply predicting the majority class.

5.3.3 Hyperparameter Tuning

An extensive hyperparameter search was conducted by training dozens of models with different configurations. The hyperparameters tuned included:

- **LSTM units:** Number of memory cells in LSTM layers
- **Dropout rate:** Regularization to prevent overfitting
- **Dense units:** Size of fully connected layers
- **Dense activation:** Activation function for dense layers

5.3.4 Model Selection Strategy

The model selection process followed a rigorous validation protocol:

1. Each candidate model was trained on the training set (60%) for 100 epochs
2. After each epoch, the model was validated against the validation set (20%)
3. Validation accuracy was tracked to identify the optimal epoch (where validation accuracy peaks)
4. This early stopping approach prevents overfitting by halting at the optimal generalization point
5. The hyperparameter configuration achieving the best performance on the validation set was selected

5.3.5 Final Model Training

Once the optimal hyperparameters and epoch count were identified, the final model was trained using a combined dataset:

1. **Combined training data:** Train + Validation sets (80% total) were merged
2. **Fresh model initialization:** A new model with the optimal hyperparameter configuration was created
3. **Optimal epoch training:** The model was trained for exactly `best_epoch` epochs (the epoch number where validation accuracy peaked)

This approach provides the model with maximum training data (80% instead of 60%) while using the optimal stopping point determined during hyperparameter tuning, achieving the best balance between learning capacity and generalization.

5.4 Results

The final trained model achieved an **accuracy of 89.56%** on the held-out test set (20% of data), demonstrating strong generalization to unseen riding scenarios.

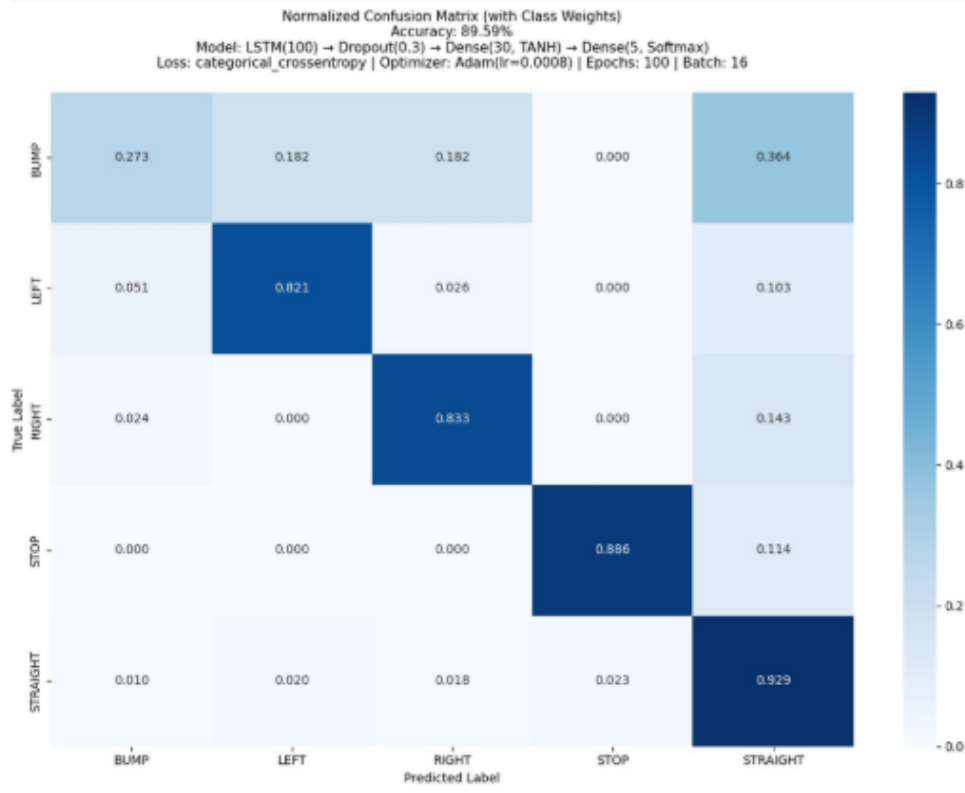


Figure 5.1: LSTM Model Confusion Matrix

5.5 Advantages of Using Class Weights

The implementation of class weights during training provided several critical benefits:

1. **Improved Minority Class Performance:** The weighted loss function penalizes misclassification of minority classes more heavily, boosting the model's focus on underrepresented groups like BUMP and STOP events.
2. **Reduced Majority Bias:** Class weights adjust the loss function to counteract the model's natural tendency toward favoritism of the majority STRAIGHT class, ensuring balanced learning across all event types.
3. **Better Generalization:** By forcing the model to build stronger decision boundaries for minority classes, the overall classification performance improves, especially for rare but critical events.
4. **Efficient Training:** Class weighting enhances fairness without requiring data re-sampling techniques (oversampling/undersampling), saving computational resources and avoiding potential issues with synthetic data generation.

The combination of LSTM's temporal modeling capability, careful hyperparameter tuning, and class weight balancing resulted in a robust event classification system capable of real-time operation on edge devices.

Chapter 6

Driver Score Generation

6.1 Overview

The driver scoring system provides real-time evaluation of driving behavior, generating objective safety scores ranging from 0 to 100, which are also represented as star ratings from 0 to 5 stars (). This quantitative assessment enables fair comparison between riders and helps identify areas for improvement.

6.1.1 Data Source

The scoring algorithm processes sensor data continuously:

- **Sensor inputs:** Accelerometer and gyroscope data from MPU6500, GPS speed
- **Processing frequency:** Data is evaluated every second
- **Additional inputs:** LSTM inference predictions and the warning list generated by the detection algorithms

6.1.2 Scoring Approach

The system employs a **penalty-based approach with rewards for safe driving**:

- **Starting point:** Base score of 100
- **Penalties:** Deducted for dangerous driving behaviors
- **Bonuses:** Added for sustained safe driving
- **Final range:** Clamped between 0 and 100

6.2 Metrics Classification

6.2.1 Bad Driving Metrics (Penalties)

The following behaviors result in score penalties:

1. **Overspeeding:** Exceeding posted speed limits

2. **Harsh Braking:** Abrupt deceleration events
3. **Bad Bump Handling:** Improper navigation of speed bumps
4. **Bad Pothole Handling:** Inadequate pothole avoidance
5. **Sudden Acceleration:** Aggressive acceleration patterns
6. **Speedy Turns:** Taking turns at excessive speeds

6.2.2 Good Driving Metric (Bonus)

Safe driving behavior is rewarded:

- **Zero-warning data points:** Time periods with no active warnings indicate safe driving behavior and contribute to a positive bonus

6.3 Penalty Calculation

6.3.1 Dynamic Penalty Formula

For each warning type (except overspeeding, which has special handling), the penalty is calculated as:

$$\text{Penalty} = \frac{\text{Warning Count}}{\text{Total Data Points}} \times W_{\text{risk}} \quad (6.1)$$

where W_{risk} is the risk weight assigned to that particular warning type based on its severity.

6.3.2 Special Case: Overspeeding Amplification

Overspeeding receives enhanced penalties that increase with the magnitude of speed violations, recognizing that higher speeds pose exponentially greater risks.

Step 1: Calculate Overspeed Magnitude

$$\text{Overspeed Magnitude} = \frac{\text{Actual Speed} - \text{Speed Limit}}{\text{Speed Limit}} \quad (6.2)$$

Step 2: Calculate Average Magnitude

$$\text{Average Magnitude} = \frac{\sum (\text{Overspeed Magnitudes})}{\text{Overspeed Count}} \quad (6.3)$$

Step 3: Calculate Base Penalty

$$\text{Base Penalty} = \frac{\text{Overspeed Count}}{\text{Total Points}} \times W_{\text{high}} \quad (6.4)$$

Step 4: Apply Amplification

$$\text{Amplified Penalty} = \text{Base Penalty} \times (1 + \text{Average Magnitude}) \quad (6.5)$$

Step 5: Calculate Extra Penalty

$$\text{Extra Penalty} = \text{Amplified Penalty} - \text{Base Penalty} \quad (6.6)$$

This amplification ensures that egregious speed violations (e.g., going 80 km/h in a 40 km/h zone) are penalized far more severely than minor infractions.

6.4 Positive Bonus Calculation

Safe driving periods without any warnings contribute a positive bonus to the final score:

Step 1: Calculate Good Ratio

$$\text{Good Ratio} = \frac{\text{Good Data Points}}{\text{Total Data Points}} \quad (6.7)$$

Step 2: Apply Bonus with Cap

$$\text{Positive Bonus} = \min(\text{Good Ratio} \times W_{\text{bonus}}, W_{\text{bonus}}) \quad (6.8)$$

The minimum function ensures the bonus does not exceed the predefined weight W_{bonus} , maintaining balance in the scoring system.

6.5 Final Score Computation

6.5.1 Total Negative Effect

All penalties are summed and capped at a maximum negative effect:

Step 1: Sum All Penalties

$$\text{Raw Negative} = \sum (\text{All Warning Penalties}) + \text{Overspeeding Extra Penalty} \quad (6.9)$$

Step 2: Apply Negative Cap

$$\text{Negative Effect} = \min(\text{Raw Negative}, N_{\text{max}}) \quad (6.10)$$

where N_{max} is the maximum allowed negative impact, preventing a single bad ride from completely destroying the score.

6.5.2 Final Score Formula

The final driver score combines penalties and bonuses with boundary constraints:

$$\text{Final Score} = \text{clamp}(100 - \text{Negative Effect} + \text{Positive Bonus}, 0, 100) \quad (6.11)$$

where the clamp function ensures:

$$0 \leq \text{Final Score} \leq 100 \quad (6.12)$$

Safety Score - Advanced Settings Back

Risk Categories
Assign each warning as High or Low risk.

Overspeeding: Low risk Speedy Turns: High risk

Harsh Braking: High risk Sudden Accel: Low risk

Bad Bump Handling: High risk Bad Pothole Handling: High risk

Weights

High Risk Weight: 30

Low Risk Weight: 15

Positive Weight: 20

Global Settings

Bump Window: 5 Cluster Window: 3

Negative Max: 70

Save Reset draft Rows: 0

Figure 6.1: Driver Score Calculation Flow

6.5.3 Score Interpretation

The numerical score is translated to a star rating for intuitive understanding:

Score Range	Interpretation
0-20	Very Poor
21-40	Poor
41-60	Average
61-80	Good
81-100	Excellent

This scoring system provides objective, data-driven feedback to riders while maintaining fairness through balanced penalties and rewards. The amplified overspeeding penalty ensures that the most dangerous behavior receives appropriate weight, while the positive bonus encourages sustained safe driving practices.

Chapter 7

Software Stack

7.1 Overview

The system’s software stack comprises two primary user-facing applications: a Rider Mobile Application for real-time feedback during rides, and a Fleet Manager Dashboard for comprehensive post-ride analysis and fleet-wide monitoring.

7.2 Dual-Strategy Data Approach

The system employs a dual-strategy approach for data transmission, optimizing both real-time responsiveness and comprehensive post-ride analytics.

7.2.1 Live Status Updates (During the Ride)

To provide immediate feedback and warnings to the rider, the system continuously pushes live snapshots of the current ride state to the driver’s mobile application. This real-time data stream includes:

- **Current Speed:** The vehicle’s instantaneous speed
- **Speed Limit:** The posted speed limit for the current location (from OLA Maps API)
- **Active Warnings:** A list of currently triggered warnings (e.g., "Overspeeding", "Pothole Ahead", "Harsh Braking")
- **Latest LSTM Event Prediction:** The most recent event classification (BUMP, LEFT, RIGHT, STOP, STRAIGHT)

These updates enable the rider to respond immediately to hazards and adjust their driving behavior in real-time.

7.2.2 Complete Ride Log (After the Ride Ends)

Once a ride concludes, the entire ride statistics are uploaded to the Fleet Manager Dashboard for comprehensive analysis. The complete `warnings_{ride_id}.csv` file is transmitted, containing every single data point collected during the ride:

- **MPU values:** All 6-axis accelerometer and gyroscope readings (accel_x, accel_y, accel_z, gyro_x, gyro_y, gyro_z)
- **GPS coordinates:** Latitude and longitude for route tracking
- **Speed data:** Actual speed and corresponding speed limit at each timestamp
- **LSTM predictions:** Event classification for each processed batch
- **Warnings:** All triggered warnings with their timestamps and contexts

7.3 Rider Mobile Application

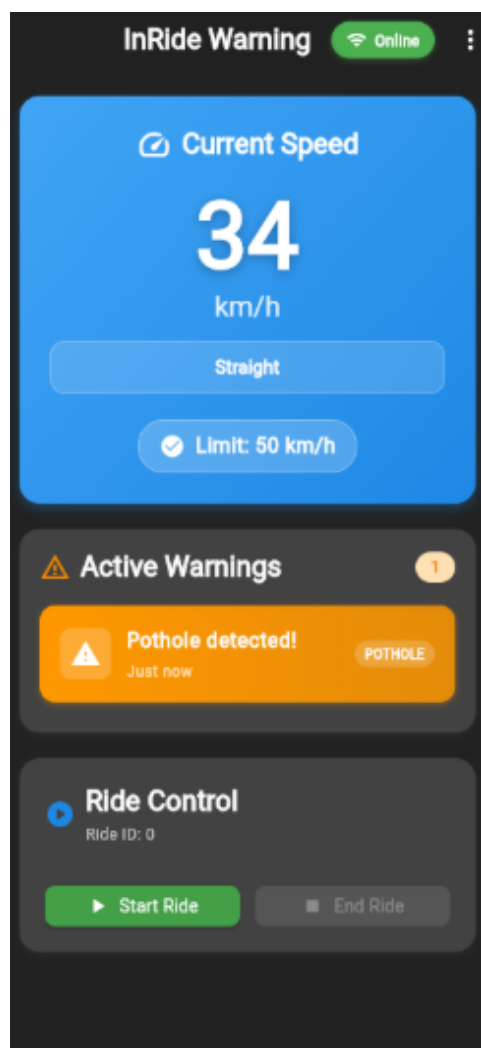


Figure 7.1: Rider Mobile Application Dashboard

The rider application focuses on real-time safety and feedback:

- **Live Dashboard:** Displays current speed, speed limit, and active warnings with visual/audio alerts
- **Immediate Warnings:** Alerts for dangerous behaviors and road hazards as they occur
- **Ride Progress:** Real-time tracking of ride duration and basic statistics

7.4 Fleet Manager Dashboard

The fleet manager dashboard provides comprehensive analytical capabilities:

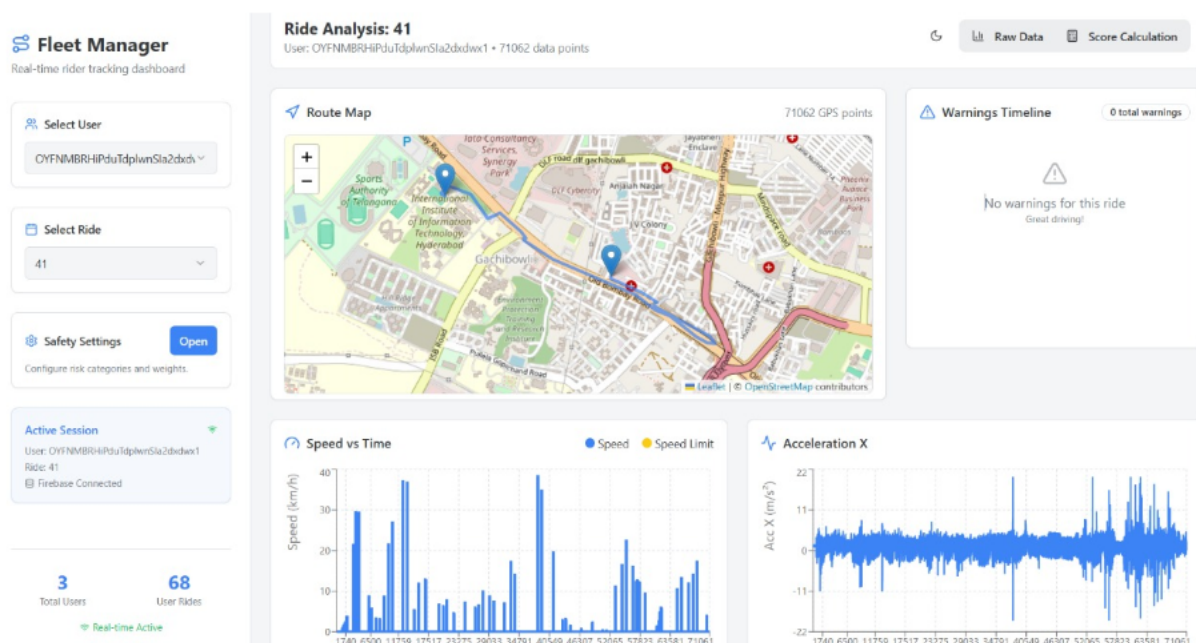


Figure 7.2: Fleet Manager Admin Dashboard

- **Complete Ride History:** Access to full CSV logs for every ride with granular sensor data
- **Driver Scoring:** View calculated safety scores and star ratings for each rider
- **Event Analytics:** Statistical analysis of warning frequencies, event distributions, and behavioral patterns



Figure 7.3: Analytics and Graphs on Admin Dashboard

- **Route Visualization:** Map-based display of routes with overlaid warning markers and speed profiles
- **Fleet Comparison:** Comparative rankings and performance metrics across multiple riders

This dual-strategy architecture ensures that riders receive immediate, actionable feedback during their journey, while fleet managers gain access to comprehensive historical data for long-term analysis, training, and optimization.

Chapter 8

Conclusions and Future Work

8.1 Conclusions

This project successfully demonstrates a comprehensive, end-to-end embedded system for two-wheeler event detection and safety monitoring. By integrating multiple sensor modalities—including a 6-axis IMU, GPS module, and camera—with advanced machine learning algorithms and real-time warning systems, we have created a practical solution that addresses critical challenges in fleet management and rider safety. The system’s key achievements include:

- **High-frequency data acquisition:** Reliable 104 Hz sampling with multi-threaded sensor fusion and Kalman filtering for noise reduction
- **Accurate event classification:** LSTM model achieving 89.56% accuracy on real-world driving events through careful handling of class imbalance and hyperparameter optimization
- **Real-time warning generation:** Multi-threaded detection algorithms for overspeeding, harsh braking, sudden acceleration, speedy turns, and road hazards
- **Computer vision integration:** Efficient pothole and speed bump detection running at 1.5-2 Hz on edge hardware
- **Objective driver scoring:** Fair, penalty-based scoring system (0-100 scale) with amplified penalties for dangerous behaviors
- **Dual-strategy data transmission:** Live updates for immediate rider feedback and complete ride logs for comprehensive fleet analysis

The system runs entirely on affordable Raspberry Pi hardware, making it cost-effective and scalable for deployment across large fleets. The combination of real-time monitoring, data-driven insights, and cloud connectivity provides actionable intelligence that can improve rider safety, reduce operational costs, and enable predictive maintenance.

8.2 Future Work

While the current system demonstrates strong performance, several avenues for improvement and expansion have been identified.

8.2.1 Model Improvements

Enhanced Training

Further training of the LSTM model on significantly larger datasets with improved training algorithms can boost classification accuracy:

- Collect additional labeled data points to expand the training corpus beyond 2.7 lakh samples
- Experiment with advanced optimization techniques (e.g., AdamW, learning rate scheduling, gradient clipping)
- Explore ensemble methods combining multiple LSTM variants for improved robustness

Diverse Dataset Collection

Training on diverse datasets across different operating conditions will improve model generalizability:

- **Vehicle types:** Scooters, motorcycles, electric bikes with varying weight distributions and dynamics
- **Road conditions:** Smooth highways, rough urban roads, unpaved surfaces, construction zones
- **Geographies:** Different regions with unique driving patterns and infrastructure characteristics
- **Weather conditions:** Rain, fog, extreme temperatures affecting sensor behavior and road traction

This broader data coverage will enable the system to perform reliably across varied deployment scenarios.

Adaptive Learning

Implement online learning capabilities to allow the model to adapt to new driving behaviors and environments:

- Continuous learning from field data to capture emerging patterns
- Personalized models that adapt to individual rider characteristics
- Automatic drift detection and model retraining triggers

Cloud-Based Training Infrastructure

Scale training and inference using cloud platforms for faster development cycles:

- Leverage GPU clusters for hyperparameter search and model training
- Implement distributed training for handling massive datasets
- Deploy model versioning and A/B testing infrastructure for continuous improvement

8.2.2 System Improvements and Scalability

Edge Deployment Optimization

Optimize the system for broader deployment:

- Deploy edge computing nodes closer to data sources for reduced latency
- Implement efficient model compression techniques (quantization, pruning) for faster inference
- Develop over-the-air (OTA) update mechanisms for seamless firmware and model upgrades

Advanced Safety Features

Integrate additional sensors and algorithms for enhanced safety:

- **Distance monitoring:** Calculate distance from other vehicles using ultrasonic sensors, LIDAR, or computer vision for collision warning systems
- **Blind spot detection:** Monitor adjacent lanes for safer lane changes and turns
- **Forward collision warning:** Predict potential frontal collisions based on relative velocity and distance

Environmental Robustness

Improve performance across diverse environmental conditions:

- **Weather adaptation:** Adjust detection thresholds and algorithms for rain, fog, and low-visibility conditions
- **Night operation:** Enhance camera-based detection with infrared or improved low-light image processing
- **Sensor fusion enhancement:** Dynamically weight sensor inputs based on environmental reliability (e.g., prioritize IMU over camera in fog)

Privacy and Security Enhancements

Implement robust privacy protections for rider data:

- **Coordinate anonymization:** Hash or spatially blur GPS coordinates to protect exact location privacy
- **End-to-end encryption:** Encrypt all data uploads to Firebase using industry-standard protocols
- **Data retention policies:** Implement automatic deletion of detailed sensor logs after a specified period (e.g., 90 days), retaining only aggregate statistics
- **User consent management:** Provide granular controls for data sharing and analytics participation

8.3 Closing Remarks

This project represents a significant step toward safer, more efficient two-wheeler fleet operations through the integration of embedded systems, machine learning, and cloud computing. The modular architecture and scalable design provide a solid foundation for future enhancements, enabling the system to evolve alongside emerging technologies and changing operational requirements. By addressing the identified areas for improvement—particularly in model training, environmental robustness, and privacy protection—this system can become a comprehensive solution deployable across diverse global markets, ultimately contributing to reduced accidents, improved rider behavior, and more sustainable fleet operations.